

Technical Design Document: Facebook Backend Platform — Registration, Authentication & News Feed Pipeline

Document Version: 2.0.0
Status: Ready for Implementation
Category: Backend Systems / Platform Infrastructure

1. Executive Summary

This document specifies the backend systems powering Facebook's core user-facing flows: user registration (sign-up), authentication (login/session), and news feed delivery. It covers the database schemas, API contracts, validation logic, security constraints, and performance requirements for each subsystem. The system must handle **50,000 concurrent registrations per minute** at peak, authenticate users within **150ms**, and deliver a personalized news feed within **500ms** of request.

2. Module 1: User Registration & Sign-Up (REG)

2.1 Data Schema

Table: tblUsers

Column	Type	Description
UserID	BIGINT AUTO_INCREMENT	Primary key
Email	VARCHAR(254)	Unique, normalized to lowercase
PhoneNumber	VARCHAR(20)	E.164 format, nullable
PasswordHash	VARCHAR(256)	bcrypt hash, cost factor 12
FirstName	VARCHAR(50)	Required, 2-50 chars
LastName	VARCHAR(50)	Required, 2-50 chars
DateOfBirth	DATE	Must be >= 13 years old
Gender	ENUM	Values: MALE, FEMALE, CUSTOM, PREFER_NOT_TO_SAY
AccountStatus	ENUM	Values: PENDING_VERIFICATION, ACTIVE, SUSPENDED, DEACTIVATED, DELETED
CreatedAt	TIMESTAMP(3)	Registration timestamp with millisecond precision
UpdatedAt	TIMESTAMP(3)	Last modification timestamp

Column	Type	Description
VerificationToken	VARCHAR(128)	SHA-512 token for email/phone verification
TokenExpiresAt	TIMESTAMP(3)	Verification token expiry (24 hours from creation)
RegistrationIP	VARCHAR(45)	IPv4 or IPv6 address of registration request
RegistrationUserAgent	VARCHAR(500)	Browser/device user agent string
Locale	VARCHAR(10)	e.g., 'en_US', 'hi_IN'

Table: tblRegistrationAttempts

Column	Type	Description
AttemptID	UUID	Primary key
Email	VARCHAR(254)	Email used in the attempt
IPAddress	VARCHAR(45)	Source IP
AttemptedAt	TIMESTAMP(3)	When the attempt occurred
Result	ENUM	Values: SUCCESS, DUPLICATE_EMAIL, INVALID_AGE, RATE_LIMITED, CAPTCHA_FAILED, BLOCKED_IP
FailureReason	TEXT	Detailed failure description (nullable)

2.2 Registration API Contract

Endpoint: POST /api/v2/auth/register

Request Body:

```
{
  "email": "string (required, valid email format)",
  "password": "string (required, 8-128 chars, must contain uppercase + lowercase + digit + special char)",
  "firstName": "string (required, 2-50 chars, no digits allowed)",
  "lastName": "string (required, 2-50 chars, no digits allowed)",
  "dateOfBirth": "YYYY-MM-DD (required, age >= 13)",
  "gender": "MALE | FEMALE | CUSTOM | PREFER_NOT_TO_SAY (required)",
  "phoneNumber": "string (optional, E.164 format)",
  "locale": "string (optional, default en_US)",
  "captchaToken": "string (required, reCAPTCHA v3 token)"
}
```

Response (201 Created):

```
{
```

```

"userId": "bigint",
"email": "string",
"accountStatus": "PENDING_VERIFICATION",
"verificationMethod": "EMAIL | SMS",
"message": "Verification link sent to your email"
}

```

Error Responses:

- 400 Bad Request — validation failure (missing fields, invalid format)
- 409 Conflict — email already registered
- 422 Unprocessable Entity — age under 13
- 429 Too Many Requests — rate limit exceeded (max 5 registration attempts per IP per hour)

2.3 Registration Validation Logic

REG-1: Email Uniqueness Check

```

SELECT COUNT(*) AS ExistingCount
FROM tblUsers
WHERE Email = LOWER(@Email)
AND AccountStatus NOT IN ('DELETED');

```

- **Assertion:** ExistingCount = 0 before proceeding.
- **Failure:** Return 409 Conflict with {"error": "EMAIL_ALREADY_REGISTERED"}.

REG-2: Age Verification

```

SELECT DATEDIFF(YEAR, @DateOfBirth, GETDATE()) AS Age;

```

- **Assertion:** Age >= 13
- **Failure:** Return 422 with {"error": "MINIMUM_AGE_NOT_MET", "minimumAge": 13}
- **Regulatory note:** Users aged 13-17 are automatically assigned SafetyTier = 'MINOR' in tblUserSafetyProfile.

REG-3: Password Strength Validation

Password must satisfy ALL of:

- Length: 8-128 characters
- Contains at least 1 uppercase letter: [A-Z]
- Contains at least 1 lowercase letter: [a-z]
- Contains at least 1 digit: [0-9]
- Contains at least 1 special character: [!@#\$%^&*()_+~\-=\[\]\{\};':"\|,.<>\/?]
- NOT in the top 10,000 common passwords list (Redis SET: common_passwords)

- NOT containing the user's first name, last name, or email prefix

REG-4: Rate Limiting

```
Redis command: INCR reg_attempts:{IP_ADDRESS}
Redis command: EXPIRE reg_attempts:{IP_ADDRESS} 3600
```

- **Assertion:** `reg_attempts:{IP_ADDRESS} <= 5`
- **Failure:** Return 429 with `Retry-After: {seconds_remaining}` header.

REG-5: CAPTCHA Verification

POST to Google reCAPTCHA v3 API:

```
POST https://www.google.com/recaptcha/api/siteverify
Body: secret={SECRET_KEY}&response={captchaToken}
```

- **Assertion:** `Response success = true AND score >= 0.5`
- **Failure:** Return 400 with `{"error": "CAPTCHA_VERIFICATION_FAILED"}`

2.4 Registration Processing Pipeline

After validation passes:

1. **Hash password:** `bcrypt.hash(password, costFactor=12)`
2. **Generate verification token:** `crypto.randomBytes(64).toString('hex')` — produces 128-char SHA-512 token
3. **Set token expiry:** `NOW() + INTERVAL 24 HOUR`
4. **Insert into `tblUsers`** with `AccountStatus = 'PENDING_VERIFICATION'`
5. **Log attempt** in `tblRegistrationAttempts` with `Result = 'SUCCESS'`
6. **Send verification email/SMS** via the Notification Service queue (Kafka topic: `user.verification.events`)
7. **Publish event** to Kafka topic `user.registration.events` for downstream consumers (analytics, onboarding flow)

2.5 Email Verification

Endpoint: `GET /api/v2/auth/verify?token={verificationToken}&userId={userId}`

```
SELECT UserID, VerificationToken, TokenExpiresAt, AccountStatus
FROM tblUsers
WHERE UserID = @UserID
  AND VerificationToken = @Token
  AND TokenExpiresAt > NOW()
  AND AccountStatus = 'PENDING_VERIFICATION';
```

- **Success:** Update `AccountStatus = 'ACTIVE'`, clear `VerificationToken`, return 200 with redirect URL.

- **Expired token:** Return 410 Gone with option to resend verification.
 - **Invalid token:** Return 400 Bad Request, log as suspicious in tblRegistrationAttempts.
-

3. Module 2: Authentication & Session Management (AUTH)

3.1 Data Schema

Table: tblSessions

Column	Type	Description
SessionID	UUID	Primary key
UserID	BIGINT	FK to tblUsers
AccessToken	VARCHAR(512)	JWT access token
RefreshToken	VARCHAR(256)	Opaque refresh token
DeviceFingerprint	VARCHAR(256)	SHA-256 hash of device attributes
IPAddress	VARCHAR(45)	Login IP
UserAgent	VARCHAR(500)	Browser/device info
CreatedAt	TIMESTAMP(3)	Session start
ExpiresAt	TIMESTAMP(3)	Access token expiry (15 minutes)
RefreshExpiresAt	TIMESTAMP(3)	Refresh token expiry (30 days)
IsRevoked	BOOLEAN	Whether session was explicitly revoked
LastActivityAt	TIMESTAMP(3)	Last API call using this session

Table: tblLoginAttempts

Column	Type	Description
AttemptID	UUID	Primary key
Email	VARCHAR(254)	Email used
IPAddress	VARCHAR(45)	Source IP
AttemptedAt	TIMESTAMP(3)	Timestamp
Result	ENUM	SUCCESS, INVALID_CREDENTIALS, ACCOUNT_LOCKED, MFA_REQUIRED, MFA_FAILED
DeviceFingerprint	VARCHAR(256)	Device hash

3.2 Login API Contract

Endpoint: POST /api/v2/auth/login

Request Body:

```
{
  "email": "string (required)",
  "password": "string (required)",
  "deviceFingerprint": "string (required)",
  "mfaCode": "string (optional, 6-digit TOTP)"
}
```

Response (200 OK):

```
{
  "accessToken": "JWT string (expires in 15 minutes)",
  "refreshToken": "opaque string (expires in 30 days)",
  "userId": "bigint",
  "expiresIn": 900,
  "tokenType": "Bearer"
}
```

3.3 Authentication Logic

AUTH-1: Credential Verification

```
SELECT UserID, PasswordHash, AccountStatus, Email
FROM tblUsers
WHERE Email = LOWER(@Email)
      AND AccountStatus IN ('ACTIVE', 'PENDING_VERIFICATION');
```

Then: `bcrypt.compare(@Password, PasswordHash)` — must return `true`.

- **Failure (wrong password):** Increment `login_failures:{Email}` in Redis. Log in `tblLoginAttempts`.

AUTH-2: Account Lockout

Redis command: `GET login_failures:{Email}`

- **Assertion:** `login_failures:{Email} < 5`
- **Lockout trigger:** After 5 consecutive failed attempts, lock the account for 30 minutes.
- **Lockout implementation:** `SET account_locked:{Email} 1 EX 1800`
- **Check before auth:** If `account_locked:{Email}` exists, return 423 Locked with `Retry-After: {seconds}`.

AUTH-3: JWT Access Token Generation

```
{
  "header": {"alg": "RS256", "typ": "JWT"},
  "payload": {
    "sub": "{UserID}",
    "email": "{Email}",
    "iat": "{unix_timestamp}",
    "exp": "{unix_timestamp + 900}",
    "iss": "facebook.com",
  }
}
```

```
"aud": "facebook-api",
"sessionId": "{SessionID}",
"safetyTier": "{SafetyTier}"
}
}
```

- Signed with RSA-256 private key
- Access token TTL: **15 minutes** (900 seconds)
- Refresh token TTL: **30 days**

AUTH-4: Concurrent Session Limit

```
SELECT COUNT(*) AS ActiveSessions
FROM tblSessions
WHERE UserID = @UserID
      AND IsRevoked = 0
      AND RefreshExpiresAt > NOW();
```

- **Assertion:** ActiveSessions <= 5
- **Action if exceeded:** Revoke the oldest session (UPDATE tblSessions SET IsRevoked = 1 WHERE SessionID = @OldestSessionID)

AUTH-5: Token Refresh

Endpoint: POST /api/v2/auth/refresh

```
SELECT SessionID, UserID, RefreshToken, RefreshExpiresAt, IsRevoked
FROM tblSessions
WHERE RefreshToken = @RefreshToken
      AND IsRevoked = 0
      AND RefreshExpiresAt > NOW();
```

- **Success:** Issue new access token, update LastActivityAt.
- **Failure (expired/revoked):** Return 401 Unauthorized, force re-login.
- **Rotation:** Generate new refresh token, revoke old one. This prevents refresh token reuse.

4. Module 3: News Feed Delivery Pipeline (FEED)

4.1 Data Schema

Table: tblFeedRankings

Column	Type	Description
RankingID	UUID	Unique identifier for each ranking event
UserID	BIGINT	The user whose feed is being ranked
PostID	BIGINT	The content item being ranked
RankScore	DECIMAL(10,6)	Computed ranking score

Column	Type	Description
		(0.000000 to 1.000000)
Position	INT	Final position in the feed (1 = top)
ContentType	ENUM	ORGANIC, SPONSORED, SUGGESTED, GROUP, PAGE, STORY
AuthorID	BIGINT	Creator of the content
EngagementPrediction	DECIMAL(8,4)	Predicted engagement probability
IntegrityScore	DECIMAL(6,4)	Content safety score (0.0000 to 1.0000)
RankedAt	TIMESTAMP(3)	When the ranking was computed
SessionID	UUID	User session that triggered the ranking

Table: tblPosts

Column	Type	Description
PostID	BIGINT	Primary key
AuthorID	BIGINT	FK to tblUsers
ContentText	TEXT	Post text content (max 63,206 chars)
MediaURLs	JSON	Array of media attachment URLs
PostType	ENUM	TEXT, IMAGE, VIDEO, LINK, STORY, REEL
Privacy	ENUM	PUBLIC, FRIENDS, FRIENDS_OF_FRIENDS, ONLY_ME, CUSTOM
CreatedAt	TIMESTAMP(3)	Post creation timestamp
IsDeleted	BOOLEAN	Soft delete flag
IntegrityScore	DECIMAL(6,4)	ML-computed safety score

Table: tblFriendships

Column	Type	Description
FriendshipID	BIGINT	Primary key
UserID1	BIGINT	Smaller UserID (canonical ordering)
UserID2	BIGINT	Larger UserID

Column	Type	Description
Status	ENUM	PENDING, ACCEPTED, BLOCKED
CreatedAt	TIMESTAMP(3)	When friendship was established

4.2 Feed Generation Pipeline

Endpoint: GET /api/v2/feed?cursor={cursor}&limit={limit}

Step 1 — Candidate Retrieval (FEED-1)

Fetch eligible posts from friends and followed pages:

```
SELECT p.PostID, p.AuthorID, p.ContentText, p.PostType, p.Privacy,
       p.CreatedAt, p.IntegrityScore, p.MediaURLs
FROM tblPosts p
INNER JOIN tblFriendships f
  ON (f.UserID1 = @UserID AND f.UserID2 = p.AuthorID AND f.Status =
'ACCEPTED')
  OR (f.UserID2 = @UserID AND f.UserID1 = p.AuthorID AND f.Status =
'ACCEPTED')
WHERE p.IsDeleted = 0
      AND p.CreatedAt > NOW() - INTERVAL 7 DAY
      AND p.Privacy IN ('PUBLIC', 'FRIENDS')
ORDER BY p.CreatedAt DESC
LIMIT 500;
```

- **Candidate pool:** Maximum 500 posts from the last 7 days.
- **Privacy filter:** Only PUBLIC and FRIENDS posts for accepted friendships.

Step 2 — ML Ranking (FEED-2)

Pass candidates to the ranking model. Each post receives:

- RankScore = weighted combination of:
 - EngagementPrediction × 0.40 (predicted likes/comments/shares)
 - RecencyScore × 0.25 (exponential decay: $e^{(-\lambda \times \text{hours_since_post})}$ where $\lambda = 0.1$)
 - RelationshipScore × 0.20 (interaction frequency with author)
 - IntegrityScore × 0.15 (content safety)

Formula:

$$\text{RankScore} = 0.40 \times \text{EngagementPrediction} + 0.25 \times e^{\{-0.1 \times \text{HoursSincePost}\}} + 0.20 \times \text{RelationshipScore} + 0.15 \times \text{IntegrityScore}$$

Step 3 — Integrity Validation (FEED-3)

Before serving, validate:

```
SELECT r.PostID, r.Position, r.IntegrityScore, r.ContentType,
       u.SafetyTier
```

```
FROM tblFeedRankings r
INNER JOIN tblUserSafetyProfile u ON r.UserID = u.UserID
WHERE r.SessionID = @SessionID
      AND r.Position <= 20
      AND (
        r.IntegrityScore < 0.3000
        OR (u.SafetyTier IN ('MINOR', 'VULNERABLE') AND r.IntegrityScore <
0.8000)
      );
```

- **Assertion:** Zero rows returned — no unsafe content in top 20.
- **Violation:** Remove violating posts, re-rank remaining, log to tblRankingAuditLog.

Step 4 — Sponsored Content Ratio Check (FEED-4)

```
SELECT
  COUNT(CASE WHEN ContentType = 'SPONSORED' THEN 1 END) AS SponsoredCount,
  COUNT(*) AS TotalCount,
  CAST(COUNT(CASE WHEN ContentType = 'SPONSORED' THEN 1 END) AS FLOAT) /
COUNT(*) * 100 AS SponsoredPercent
FROM tblFeedRankings
WHERE SessionID = @SessionID AND Position <= 20;
```

- **Assertion:** SponsoredPercent <= 25.0
- **Violation:** Remove excess sponsored posts, fill with organic content.

Step 5 — Diversity Check (FEED-5)

```
SELECT
  COUNT(DISTINCT AuthorID) AS UniqueAuthors,
  COUNT(DISTINCT ContentType) AS UniqueContentTypes
FROM tblFeedRankings
WHERE SessionID = @SessionID AND Position <= 20;
```

- **Assertion:** UniqueAuthors >= 5 AND UniqueContentTypes >= 3

4.3 Feed Pagination

Uses cursor-based pagination:

- cursor = encrypted {lastPostID}:{lastRankScore} from previous page
- limit = 10-50 (default 20)
- Next page query appends: WHERE RankScore < @LastRankScore OR (RankScore = @LastRankScore AND PostID < @LastPostID)

5. Cross-Cutting Concerns

5.1 Security Requirements

- All passwords stored as **bcrypt** hashes with cost factor **12**
- All API endpoints require **HTTPS** (TLS 1.3)
- JWT tokens signed with **RS256** (2048-bit RSA key pair, rotated every 90 days)

- PII (email, phone, IP) encrypted at rest using **AES-256-GCM**
- Rate limiting at API gateway level: **1000 requests per user per minute**
- CORS restricted to `*.facebook.com` origins

5.2 Audit & Compliance

Table: tblAuditLog

Column	Type	Description
AuditID	UUID	Primary key
UserID	BIGINT	Affected user
Action	VARCHAR(50)	REGISTER, LOGIN, LOGOUT, PASSWORD_CHANGE, ACCOUNT_DELETE, FEED_VIEW
IPAddress	VARCHAR(45)	Source IP
Details	JSON	Action-specific metadata
CreatedAt	TIMESTAMP(3)	When the action occurred

- Every registration, login, password change, and account deletion is logged.
- Audit logs retained for **7 years** per GDPR/regulatory requirements.
- Immutable: no UPDATE or DELETE operations permitted on tblAuditLog.

5.3 Notification Pipeline

All transactional notifications (verification emails, login alerts, security warnings) are published to Kafka:

- **Topic:** `user.notification.events`
- **Partitioning:** By UserID
- **Schema:**

```
{
  "userId": "bigint",
  "notificationType": "VERIFICATION_EMAIL | LOGIN_ALERT | SECURITY_WARNING | PASSWORD_RESET",
  "channel": "EMAIL | SMS | PUSH",
  "payload": { ... },
  "priority": "HIGH | NORMAL | LOW",
  "createdAt": "ISO-8601"
}
```

6. Performance Requirements (KPIs)

Metric	Target	Measurement
Registration API Latency	< 500ms (p95)	Time from request to response
Login API Latency	< 150ms (p95)	Time from request to JWT issued
Feed Delivery Latency	< 500ms (p95)	Time from request to ranked feed returned
Email Verification Delivery	< 30 seconds	Time from registration to email in inbox
Concurrent Registrations	50,000/minute	Peak capacity without degradation
Feed Ranking Throughput	100,000 rankings/second	ML model inference capacity
Session Token Refresh	< 50ms	Time to validate refresh token and issue new access token
Database Query Latency	< 10ms (p95)	For indexed lookups on primary tables
Uptime	99.99%	Monthly availability target

7. Error Handling & Alerting

Severity	Condition	Action
CRITICAL	Registration success rate drops below 95%	PagerDuty alert + auto-scale registration workers
CRITICAL	Login success rate drops below 98% (excluding invalid credentials)	PagerDuty alert + check database connectivity
CRITICAL	Feed delivery latency exceeds 2 seconds (p95)	PagerDuty + Slack #feed-ops + fallback to cached feed
HIGH	MINOR user served content with IntegrityScore < 0.8	Slack #trust-safety + compliance log + feed re-rank
HIGH	More than 100 account lockouts per minute from same IP range	Slack #security + potential DDoS, engage WAF
MEDIUM	Sponsored content ratio exceeds 25% in any feed	Slack #ads-integrity + adjust sponsored injection logic
LOW	Verification email delivery exceeds 60 seconds	Log for SLA monitoring

All alerts published to Kafka topic `platform.alerts` with schema:

```
{  
  "alertId": "uuid",
```

```
"severity": "CRITICAL | HIGH | MEDIUM | LOW",  
"component": "REG | AUTH | FEED",  
"description": "string",  
"metrics": { ... },  
"timestamp": "ISO-8601"  
}
```